

RestTemplate -> RestClient

[Migrating to RestClient Demo Sharing](#)

Limitation of RestTemplate

```
var response = restTemplate.exchange(url: "/phone_book", POST, request, PhoneBook.class);
```

- “Template-style” API:
 - Using methods overloading for variants
 - Method namespace gets quickly crowded
 - Hard to expand new features

```
exchange(String url, HttpMethod method, HttpEntity<?  
> requestEntity, Class<T> responseType, Object  
... uriVariables)
```

```
exchange(String url, HttpMethod method, HttpEntity<?  
> requestEntity, Class<T> responseType, Map<String,?  
> uriVariables)
```

```
exchange(String url, HttpMethod method, HttpEntity<?  
> requestEntity, ParameterizedTypeReference<T> responseType,  
Object... uriVariables)
```

```
exchange(String url, HttpMethod method, HttpEntity<?  
> requestEntity, ParameterizedTypeReference<T> responseType,  
Map<String,?> uriVariables)
```

```
exchange(URI url, HttpMethod method, HttpEntity<?  
> requestEntity, Class<T> responseType)
```

```
exchange(URI url, HttpMethod method, HttpEntity<?  
> requestEntity, ParameterizedTypeReference<T> responseType)
```

```
exchange(RequestEntity<?> entity, Class<T> responseType)
```

```
exchange(RequestEntity<?> entity,  
ParameterizedTypeReference<T> responseType)
```

RestClient: Same Infrastructure, New Abstraction

- Fluent API:
 - Method chaining
 - Easy to extend



poutsma on Jul 3, 2023

Contributor Author ...

Spring 6.1 M2 will see the introduction of `RestClient`, a synchronous HTTP client that offers an API similar to `WebClient`, using the same infrastructure (i.e. request factory, error handler, interceptors, etc) as `RestTemplate`.



```
@Override 4 usages  Li
public PhoneBook createPhoneBook(PhoneBookDto phoneBookDto) {
    return restClient.post() RequestBodyUriSpec
        .body(phoneBookDto) RequestBodySpec
        .retrieve() ResponseSpec
        .body(PhoneBook.class);
}
```

Migrating to RestClient: Gradual Fashion

- Spring doc recommends adopting RestClient in a gradual fashion.
 - First, create one or more RestClient instances from existing RestTemplate instances.
 - Then, gradually replicate your existing RestTemplate instance creations by using RestClient.Builder.

Migrating to RestClient: Gradual Fashion

- When creating RestClient from existing RestTemplate, RestClient treats the provided RestTemplate as the primary configuration source.

```
public class UrlUsingRestTemplate { new *  
  
    static void main(String[] args) { new *  
        RestTemplate restTemplate = new RestTemplateBuilder()  
            .rootUri("http://httpbin.org")  
            .build();  
  
        RestClient restClient = RestClient.builder(restTemplate)  
            .baseUrl("http://postman-echo.com")  
            .build();  
  
        String body = restClient  
            .get() RequestHeadersUriSpec<capture of ?>  
            .uri(uri: "/get") capture of ?  
            .retrieve() ResponseSpec  
            .body(String.class);  
  
        System.out.println(body);  
    }  
}
```

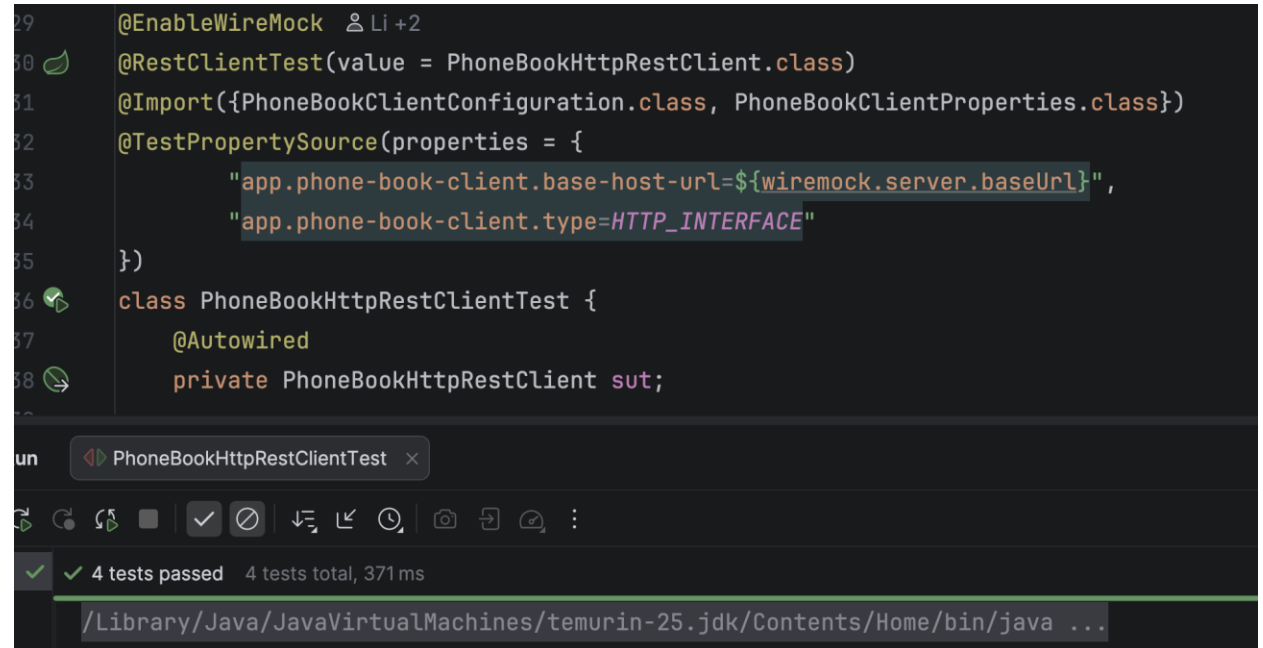
```
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin  
{  
    "args": {},  
    "headers": {  
        "Accept-Encoding": "gzip, x-gzip, deflate",  
        "Host": "httpbin.org",  
        "User-Agent": "Apache-HttpClient/5.5.2 (Java/25.0.2)",  
        "X-Amzn-Trace-Id": "Root=1-69e36240-3b76af544303a5e33e30b288"  
    },  
    "origin": "144.178.213.99",  
    "url": "http://httpbin.org/get"  
}
```

Key design shift: Post-creation mutation → Build-time ownership

- RestTemplate:
 - Mutable client
 - Configuration can be modified by framework code and test
 - Behavior is influenced by shared mutable state
- RestClient:
 - Effectively immutable after build
 - Configuration finalized at client creation time
 - Overrides are explicit and order-dependent

Interesting Diff #1 Testing

- I met an issue while testing a RestClient-backed client and a RestTemplate-backed client, where both request factories were customized.
- When I used @RestClientTest together with WireMock on the RestClient impl, everything worked as expected.



```
29 @EnableWireMock
30 @RestClientTest(value = PhoneBookHttpClient.class)
31 @Import({PhoneBookClientConfiguration.class, PhoneBookClientProperties.class})
32 @TestPropertySource(properties = {
33     "app.phone-book-client.base-host-url=${wiremock.server.baseUrl}",
34     "app.phone-book-client.type=HTTP_INTERFACE"
35 })
36 class PhoneBookHttpClientTest {
37     @Autowired
38     private PhoneBookHttpClient sut;
```

PhoneBookHttpClientTest

4 tests passed 4 tests total, 371 ms

/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...

Interesting Diff #1 Testing

- However, when I tried using `@RestClientTest` together with WireMock for testing the RestTemplate-backed client, the test failed:
 - WireMock successfully received the admin request to register the stub.
 - However, it never received the actual business HTTP call.
 - The logs indicated that the HTTP request was intercepted by `MockRestServiceServer` instead.

```
java.lang.AssertionError: No further requests expected: HTTP POST /phone_book  
0 request(s) executed.
```

```
at org.springframework.test.web.client.AbstractRequestExpectationManager.createUnexpectedRequestError(AbstractRequestExpectationManager.java:100)  
at org.springframework.test.web.client.SimpleRequestExpectationManager.matchRequest(SimpleRequestExpectationManager.java:54)  
at org.springframework.test.web.client.AbstractRequestExpectationManager.validateRequest(AbstractRequestExpectationManager.java:100)  
at org.springframework.boot.restclient.test.RootUriRequestExpectationManager.validateRequest(RootUriRequestExpectationManager.java:100)  
at org.springframework.test.web.client.MockRestServiceServer$MockClientHttpRequestFactory$1.executeInternal(MockRestServiceServer$MockClientHttpRequestFactory$1.java:100)
```

Interesting Diff #1 Testing

- @RestClientTest auto-configures MockRestServiceServer.
- MockRestServiceServer configures the client with MockClientHttpRequestFactory and intercepts requests before the HTTP call is made.
- WireMock, on the other hand, runs a real HTTP server, and the client sends real HTTP requests to that server.

```
@TypeExcludeFilters({RestClientTypeExcludeFilter.class})
@AutoConfigureMockRestServiceServer
@AutoConfigureRestClient
@ImportAutoConfiguration
public @interface RestClientTest {
```

Interesting Diff #1 Testing

- Why does `@RestClientTest + @EnableWireMock` fail for a `RestTemplate`-backed client?
 - `RestTemplate` is mutable. `MockRestServiceServer` customizes its request factory into `MockClientHttpRequestFactory`, whether or not it is explicitly configured during creation.
 - As a result, HTTP requests are intercepted by `MockRestServiceServer`.
 - However, the test didn't set up expectations for `MockRestServiceServer`, so the test failed with: "Assertion Error: no further requests expected".
- Adding `@AutoConfigureMockRestServiceServer(enabled = false)` resolved the bug.

Interesting Diff #1 Testing

- Why does @RestClientTest + @EnableWireMock work for the RestClient-backed client?
 - Calling RestClient.Builder.requestFactory() overrides MockRestServiceServer's replacement with request factory.
 - Actual HTTP requests are sent to and received by WireMock server.
- If WireMock is not enabled, the request is not intercepted by MockRestServiceServer, and the test fails with a connection error.
- If WireMock is not enabled and the request factory is not overridden, the @RestClientTest test passes, because MockRestServiceServer intercepts the request as intended.

Interesting Diff #2 URL

- RestTemplate:
 - URL resolution rule: rootUri is applied to requests that start with a '/' or are relative

```
public static RestTemplate createRestTemplate(RestTemplateBuilder restTemplateBuilder) {  
    return restTemplateBuilder  
        .requestFactory(() -> getHttpClientRequestFactory(properties))  
        .rootUri(properties.getBaseHostUrl() + "/phone_book")  
        .build();  
}
```

```
var response = restTemplate.exchange(url: null, POST, request, PhoneBook.class);  
return response.getBody();
```

Passing 'null' argument to parameter annotated as @NotNull

```
var response = restTemplate.exchange(url: "", POST, request, PhoneBook.class);
```

PhoneBookRestTemplateClientTest.createPhoneBook

1 test failed 1 test total, 212 ms

java.lang.AssertionError: Request URI expected:<http://localhost:8080/phone_book> but was:<>
Expected :http://localhost:8080/phone_book
Actual :

```
var response = restTemplate.exchange(url: "/", POST, request, PhoneBook.class);
```

PhoneBookRestTemplateClientTest.createPhoneBook

1 test failed 1 test total, 233 ms

java.lang.AssertionError: Request URI expected:<http://localhost:8080/phone_book> but was:<http://localhost:8080/phone_book/>
Expected :http://localhost:8080/phone_book
Actual :http://localhost:8080/phone_book/

Interesting Diff #2 URL

- RestClient: .uri() is optional

```
PhoneBookRestClient(RestClient.Builder restClientBuilder, PhoneBook  
    restClient = restClientBuilder  
        .baseUrl(properties.getBaseHostUrl() + "/phone_book")  
        .build();  
}
```

```
@Override 4 usages 2 Li  
public PhoneBook createPhoneBook(PhoneBookDto phoneBookDto) {  
    return restClient.post() RequestBodyUriSpec  
        .body(phoneBookDto) RequestBodySpec  
        .retrieve() ResponseSpec  
        .body(PhoneBook.class);  
}
```

```
@Override 4 usages 2 Li +1  
public PhoneBook updatePhoneBook(long id, PhoneBookDto phoneBookDto) {  
    return restClient.put() RequestBodyUriSpec  
        .uri(uri:("/{id}", id) RequestBodySpec  
        .body(phoneBookDto)  
        .retrieve() ResponseSpec  
        .body(PhoneBook.class);  
}
```

Interesting Diff #3 Error Handling

- RestTemplate
 - Uses global ResponseErrorHandler
 - Error handled centrally for all requests
- RestClient
 - Supports ResponseErrorHandler and fluent per-request status handling
 - Encourages per-request error handling
- When creating RestClient from RestTemplate:
 - Inherits configuration (including error handler)
 - Can override error handling via onStatus(...)

Interesting Diff #3 Error Handling

```
RestTemplate restTemplate = new RestTemplate();
restTemplate.setErrorHandler(new ResponseErrorHandler() { new *
    @Override new *
    public boolean hasError(@NonNull ClientHttpResponse response) throws IOException {
        return response.getStatusCode().isError();
    }

    @Override new *
    public void handleError(@NonNull URI url, @NonNull HttpMethod method, @NonNull ClientHttpResponse response) throws IOException {
        System.out.println("RestTemplate error handler triggered");
    }
});
```

```
RestClient client = RestClient.create(restTemplate);
String body = client.get() RequestHeadersUriSpec<capture of ?>
    .uri(uri: "http://httpbin.org/status/404") capture of ?
    .retrieve() ResponseSpec
    .onStatus( HttpStatusCode status -> status.value() == 404, ( HttpRequest req, ClientHttpResponse res) -> {
        System.out.println("RestClient error handler triggered");
    })
    .body(String.class);

System.out.println("Done");
```

```
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...
RestClient error handler triggered
Done
```

Interesting Diff #4 Interceptor

- RestTemplate
 - Interceptors can be added or modified after creation
 - Applied globally per client instance
- RestClient
 - Interceptors are configured at build time
 - Applied globally per client instance
- When creating RestClient from RestTemplate:
 - Existing RestTemplate interceptors are copied and chained into the RestClient

Interesting Diff #4 Interceptor

```
RestTemplate restTemplate = new RestTemplate();
restTemplate.getInterceptors().add(new ClientHttpRequestInterceptor() { new
    @Override new *
    public ClientHttpResponse intercept(@NonNull HttpRequest request, byte []
        System.out.println("RestTemplate interceptor");
        return execution.execute(request, body);
    }
});
```

```
RestClient client = RestClient.builder(restTemplate)
    .requestInterceptor((HttpRequest req, byte[] body, ClientHttpRequest
        System.out.println("RestClient interceptor");
        return exec.execute(req, body);
    })
    .build();
```

```
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...
RestTemplate interceptor
RestClient interceptor
Done

Process finished with exit code 0
```



**THANK YOU FOR YOUR
ATTENTION**